COLLEGE COMPLAINTS PORTAL

Official Comprehensive Viva-Voce & Project Defense Guide (in English)

Team: Jatin (Lead), Manish, Pradeep Kumar Meena, Pradeep, Kapil Beniwal | ACEIT, Jaipur

SECTION 1: PROJECT OVERVIEW & OBJECTIVES

Q1. What is the core objective of the College Complaints Portal project?

The College Complaints Portal is a centralized full-stack digital grievance redressal and campus maintenance management platform. Its primary objective is to streamline, digitize, and monitor student complaints regarding hostel amenities, academics, mess cleanliness, IT network, security, and anti-ragging in compliance with UGC/AICTE grievance redressal norms.

- Eliminates opaque manual paper-based grievance submission.
- Provides transparent 4-stage real-time tracking from submission to resolution.
- Introduces community upvoting so administrators can prioritize high-impact issues.

Q2. What real-world problem does this portal solve?

In traditional college administration, students face multiple challenges when filing complaints:

- Lack of Accountability: Students had no way of knowing which official had their ticket or expected turnaround time.
- Fear of Victimization: Students hesitated to report ragging or faculty issues due to identity exposure.
- No Service Level Agreement (SLA): Maintenance issues took weeks without official oversight.
- Duplication: Dozens of students filed identical complaints for common issues (e.g. library Wi-Fi down).

Q3. Who are the primary target users and stakeholders?

The system serves three primary user personas via integrated Role-Based Access:

- Students: Register tickets, choose anonymity, upvote campus feed issues, track progress, and submit 5-star resolution feedback.
- Department Officers & Technicians: View assigned issues, assign electricians/plumbers, add action remarks, and update statuses.
- Deans & College Administration: Monitor institute-wide KPIs, track satisfaction ratings, review department workloads, and export audit reports.

SECTION 2: TECHNOLOGY STACK JUSTIFICATION

Q4. Why did you select React 18 and Vite for the frontend?

React was chosen for its component-based architecture and state-driven virtual DOM rendering. Vite was selected as the build tool because:

- Extremely fast development server start using native ES modules (ESM).
- Instant Hot Module Replacement (HMR) for reactive state updates.
- Highly optimized production bundle output with dynamic asset chunking.

Q5. Why was Node.js and Express.js used for the backend?

Node.js offers an asynchronous, non-blocking, event-driven I/O model which is ideal for handling concurrent I/O operations such as complaint lookups, real-time comment threads, and file attachments with minimal memory overhead. Express provides a minimalist, robust routing middleware framework to construct clean RESTful API endpoints.

Q6. What database architecture is implemented, and how can it scale?

The current implementation utilizes a persistent JSON-based data store with atomic file operations. This zero-configuration design eliminates external server installation hurdles while maintaining full state persistence across restarts.

- REST Abstraction: Because routes interact via modular data helper functions, transitioning to MongoDB (Mongoose) or PostgreSQL (Prisma/Sequelize) requires changing only the controller queries without touching any frontend code.

Q7. Why did you choose Tailwind CSS over traditional CSS or Bootstrap?

Tailwind CSS employs a utility-first methodology. It enables building custom, modern responsive designs (glassmorphism headers, responsive grids, and animated status badges) without context-switching between CSS stylesheets, resulting in minimal CSS bundle overhead through automatic purging of unused classes.

SECTION 3: KEY FEATURES & BUSINESS LOGIC

Q8. How is student privacy and anonymity implemented?

When a student selects the "File Anonymously" option in the complaint form, the backend sets isAnonymous: true. On serialization and frontend rendering, the student's real name and roll number are suppressed and replaced with "Anonymous Student" and "HIDDEN". This protects students in anti-ragging or faculty-related matters.

Q9. Explain the Community Upvoting mechanism and its concurrency control.

Every complaint contains an "upvotedBy" array containing student identifiers (Roll Numbers). When an upvote request is received:

- The server checks if the student ID exists in the upvotedBy array.
- If present, it un-upvotes (removes ID, decrements count).
- If absent, it registers the upvote (pushes ID, increments count).
- This prevents fraudulent duplicate voting and surfaces high-impact campus issues at the top of the feed.

Q10. What is the Complaint Lifecycle and Timeline Stepper?

Every complaint moves through 4 distinct sequential states:

- 1. Submitted: Successfully recorded into the system by student.
- 2. Under Review: Acknowledged by department office or chief proctor.
- 3. In Progress: Work order dispatched to specific technician/officer.
- 4. Resolved / Rejected: Corrective action verified, or ticket dismissed with reason.
- Every status transition appends an immutable timeline event storing timestamp, actor, and official remarks.

Q11. How does the Resolution Feedback and Rating system work?

Once a ticket status reaches "Resolved", the system displays a 5-star rating widget along with a review comment box for the original student. The ratings directly aggregate into the administrator's KPI dashboard, calculating the institute's overall satisfaction index (e.g. 4.8 / 5.0).

SECTION 4: TECHNICAL & CODE IMPLEMENTATION

Q12. How is file attachment handled in the backend?

We utilized the "multer" middleware library on the /api/upload endpoint. It intercepts multipart/form-data, applies a 10MB size limit, appends a cryptographically unique timestamp suffix to avoid filename collisions, and stores files in the public server/uploads/ directory.

Q13. How does the CSV Report Export function without external libraries?

In the Admin Dashboard, client-side JavaScript extracts the filtered array of complaints, converts each record into comma-delimited strings with proper escaping for quotes, constructs a data:text/csv URI, and triggers a programmatic download via an anchor element.

Q14. What is Vite reverse proxy and why was it configured in vite.config.js?

During development, Vite runs on port 5173 while the Express API runs on port 5000. To prevent cross-origin browser CORS restrictions and cookie handling issues, Vite is configured with a proxy rule forwarding all requests starting with /api and /uploads to http://localhost:5000.

Q15. How is optimistic UI update implemented in React?

When a student clicks the Upvote button, React immediately increments the local state count and toggles the button color before the HTTP response returns. If the network request fails, the state gracefully rolls back, providing an instantaneous native-app feel.

SECTION 5: SECURITY, LIMITATIONS & FUTURE SCOPE

Q16. What security considerations were addressed in the design?

Security measures include:

- File Upload Protection: Strict file size restriction (10MB) and extension checking.
- Data Sanitization: Encoding user-generated strings to mitigate Cross-Site Scripting (XSS).
- Identity Concealment: Complete database masking for confidential anti-ragging complaints.

Q17. What are the limitations of the current prototype?

Current limitations include lack of SMS gateway integration, single-server deployment without horizontal clustering, and absence of automated speech-to-text input for accessibility.

Q18. What are the planned future enhancements?

Planned future enhancements include:

- Automated SMS / WhatsApp alerts via Twilio API when ticket status changes.
- AI-based NLP Auto-Routing to predict category and priority automatically from description.
- Geo-tagging and campus map integration using Leaflet/Google Maps API.
- Cross-platform mobile application utilizing React Native.

SECTION 6: TOP RAPID-FIRE QUESTIONS (EXAMINER CHECKLIST)

Q19. Rapid-Fire: What is REST and does this project follow REST principles?

Yes. Representational State Transfer (REST) uses standard HTTP verbs (GET, POST, PATCH) with stateless endpoints (/api/complaints, /api/stats, /api/notices) and returns standardized JSON payloads.

Q20. Rapid-Fire: Why use React hooks (useState, useEffect) instead of Class components?

Hooks allow using state and lifecycle logic without classes, making components smaller, easier to read, testable, and enabling better tree-shaking optimization in modern bundlers.

Q21. Rapid-Fire: How do you start this application from a clean clone?

Run "npm install" followed by "npm run dev" to boot both backend and frontend, or "npm start" to run the optimized single-port production server on port 5000.